چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۲۰

امنیت — بخش ۲: R8، یکپارچگی، بیومتریک، passkey و MASVS



اسماعیل مرادی

@esmaeeil-moradi700

چرا بخش دوم در ۲۰۲۶ مهم است؟

بخش اول درها را بست. بخش دوم دربارهٔ دستگاهی است که **در کنترل شما نیست** — و اینکه چطور با انتقال اعتماد به **کلیدهای سخت‌افزاری** و **اعتبارسنجی سمت سرور** باز هم امن بمانید:

- R8 هزینهٔ مهندسی معکوس را بالا می‌برد — رازها را پنهان نمی‌کند
- چک‌های root و دست‌کاری **سیگنال**‌اند، نه تصمیم
- SafetyNet Attestation بازنشسته شده و **Play Integrity** جایش را گرفته؛ key attestation دستگاه‌های بدون Play را پوشش می‌دهد
- اثر انگشت فقط وقتی معنا دارد که یک **کلید Keystore** را باز کند، یعنی با `CryptoObject`
- **passkeyها** از طریق Credential Manager دارند جای رمز عبور را می‌گیرند
- OWASP MASVS یک چک‌لیست مشترک به تیم‌ها می‌دهد؛ **زنجیرهٔ تأمین** سطح حملهٔ جدید است

سطح	سؤال‌ها
جونیور	R8 واقعاً چه می‌کند، تشخیص root و دست‌کاری، مبانی بیومتریک
مید	BiometricPrompt با CryptoObject، passkeyها، Play Integrity، صفحه‌های حساس
سینیور	اعتبارسنجی یکپارچگی روی سرور، key attestation، واکنش به دست‌کاری
آرکیتکت	دفاع در عمق، OWASP MASVS و MASTG، امنیت زنجیرهٔ تأمین



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: هزینه را بالا ببر، روی سرور تصمیم بگیر



- صاحب دستگاهی که مصمم باشد می‌تواند همه‌چیز داخل APK شما را **بخواند، patch کند و hook کند**
- چک‌های سمت کلاینت حمله را کند می‌کنند و سیگنال می‌سازند — هیچ‌وقت حرف آخر نیستند
- تضمین واقعی از **کلیدهای داخل سخت‌افزار امن** و **چک‌هایی که مهاجم نمی‌تواند دور بزند** می‌آید

تنها سؤالی که باید بپرسید

«چه کسی این را تأیید می‌کند؟» — اگر جواب «خود اپ» است، فرض کنید مهاجم می‌تواند از رویش رد شود.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

R8 واقعاً چه می‌کند، تشخیص root و دست‌کاری، مبانی بیومتریک



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. R8 واقعاً چه می‌کند

```
android {
    buildTypes {
        release {
            isMinifyEnabled = true    // shrink + obfuscate
            isShrinkResources = true
            proguardFiles(
                getDefaultProguardFile(
                    "proguard-android-optimize.txt"),
                "proguard-rules.pro",
            )
        }
    }
}
```

R8 این کارها را می‌کند	R8 این کارها را نمی‌کند
حذف کد بلااستفاده (shrinking)	رمز کردن رشته‌ها، URLها یا کلیدهای API
تغییر نام کلاس‌ها و اعضا (obfuscation)	پنهان کردن resourceها، manifest یا کتابخانه‌های native
بهینه‌سازی و inline کردن کد	جلوگیری از debugger یا Frida در زمان اجرا
ساختن `mapping.txt` برای stack trace خوانا	محافظت از منطقی که به کلاینت فرستاده‌اید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*کلید API من را پنهان می‌کند.R8*» — رشته‌های ثابت **به شکل متن ساده** در DEX می‌مانند؛ یک decompiler مثل jadx در چند دقیقه نشانشان می‌دهد. رازها جایشان روی سرور است؛ کلیدهای سمت کلاینت باید محدود شوند و عمومی فرض شوند (روز ۱۲).

تلهٔ مصاحبه

«*ما از ProGuard استفاده می‌کنیم.*» — R8 از AGP 3.4 جای ProGuard را به‌عنوان کامپایلر پیش‌فرض گرفت. همان قواعد `-keep` را می‌خواند، اما ابزار دیگری است — و از AGP 8 حالت full mode پیش‌فرض است (روز ۱۸).

سؤال تکمیلی

«*کرش production را چطور می‌خوانید؟*» — فایل `mapping.txt` هر نسخه را نگه دارید (در Play Console و crash reporter آپلودش کنید) و retrace کنید. اگر گمش کنید، stack traceهای آن نسخه برای همیشه ناخوانا می‌مانند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. تشخیص root و دست‌کاری — و محدودیت‌هایش

چک سمت کلاینت	دنبال چه می‌گردد	دور زدن رایج
نشانه‌های root	فایل‌های `su`، اپ‌های مدیریت root	ماژول‌های مخفی‌سازی root آن‌ها را از اپ‌های انتخاب‌شده پنهان می‌کنند
propertyهای build	`test-keys`، `ro.debuggable`	propertyهای جعلی
debugger و hook	`Debug.isDebuggerConnected()`، ردپای Frida	خود چک hook می‌شود
امضا و نصب‌کنندهٔ اپ	APK دوباره امضاشده یا sideloadشده	از DEX حذفش می‌کنند

همهٔ این‌ها **داخل کدی اجرا می‌شوند که در کنترل مهاجم است.** از چندتایشان استفاده کنید، نتیجه را به سرور گزارش کنید — و بگذارید سیگنال‌هایی که سرور تأیید می‌کند تصمیم بگیرند (سؤال ۶ و ۸).

تلهٔ مصاحبه

«روی دستگاه rootشده، شبیه‌ساز، VPN یا با developer options روشن اجرا نشو.» — نسخهٔ پارسال دقیقاً همین را توصیه کرده بود. VPN و developer options ابزار روزمرهٔ کاربران واقعی‌اند؛ بستنشان مشتری را تنبیه می‌کند، در حالی که مهاجم در یک بعدازظهر چک را دور می‌زند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. چک امضای خود اپ — و تله‌اش

```kotlin
fun signingCertSha256(context: Context): List<String> {
    val info = context.packageManager.getPackageInfo(
        context.packageName,
        PackageManager.GET_SIGNING_CERTIFICATES, // API 28+
    )
    val signers = info.signingInfo?.apkContentsSigners.orEmpty()
    return signers.map { sig ->
        MessageDigest.getInstance("SHA-256")
            .digest(sig.toByteArray())
            .joinToString("") { "%02x".format(it) }
    }
}
```

تلهٔ مصاحبه

«با اثر انگشت keystore خودمان مقایسه‌اش می‌کنیم.» — با **Play App Signing**، گوگل چیزی را که کاربر نصب می‌کند با **کلید امضای اپ** دوباره امضا می‌کند، نه با کلید آپلود شما. اگر کلید آپلود را hard-code کنید، چک برای همهٔ کاربران پلی شکست می‌خورد. از گواهی app signing در Play Console استفاده کنید.

تلهٔ مصاحبه

«در زمان اجرا از DEX هش بگیر و مقایسه کن.» — خود مقایسه داخل همان DEXی است که مهاجم patch می‌کند. verdict اپ در Play Integrity (سؤال ۶) این چک را جایی انجام می‌دهد که دست مهاجم به آن نمی‌رسد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. مبانی احراز هویت بیومتریک

```
val allowed = BIOMETRIC_STRONG or DEVICE_CREDENTIAL
when (BiometricManager.from(context).canAuthenticate(allowed)) {
    BiometricManager.BIOMETRIC_SUCCESS -> showPrompt()
    BiometricManager.BIOMETRIC_ERROR_NONE_ENROLLED ->
        offerEnrollment() // Settings.ACTION_BIOMETRIC_ENROLL
    else -> usePasswordInstead()
}

val info = BiometricPrompt.PromptInfo.Builder()
    .setTitle("Unlock your records")
    .setAllowedAuthenticators(allowed)
    .build() // no negative button when DEVICE_CREDENTIAL is set
```

Authenticator	معنا	مناسب برای
`BIOMETRIC_STRONG` یا Class 3	مقاومت بالا در برابر جعل؛ می‌تواند کلید Keystore را باز کند	پرداخت، دسترسی به کلید
`BIOMETRIC_WEAK` یا Class 2	ضعیف‌تر؛ با `CryptoObject` **مجاز نیست**	باز کردن قفل برای راحتی
`DEVICE_CREDENTIAL`	پین، الگو یا رمز عبور	جایگزین



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*صدا خوردن* `onAuthenticationSucceeded` *یعنی هویت کاربر تأیید شد.*» — بدون `CryptoObject` این فقط یک callback داخل پروسس شماست. یک ابزار hook می‌تواند مستقیم صدایش بزند. برای هر چیز ارزشمند، prompt را به یک کلید گره بزنید (سؤال ۴).

تلهٔ مصاحبه

«*ما از* `FingerprintManager` *استفاده می‌کنیم.*» — از API 28 منسوخ شده. `BiometricPrompt` در AndroidX چهره و روش‌های دیگر را هم پوشش می‌دهد، UI سیستمی دارد و برای نسخه‌های قدیمی‌تر backport شده.

سؤال تکمیلی

«*آیا همهٔ نسخه‌ها ترکیب* `BIOMETRIC_STRONG` *و* `DEVICE_CREDENTIAL` *را پشتیبانی می‌کنند؟*» — نه. این ترکیب روی API 28 و 29 پشتیبانی نمی‌شود، و `DEVICE_CREDENTIAL` به‌تنهایی API 30 به بالا می‌خواهد. به‌جای فرض کردن، با `canAuthenticate()` چک کنید.



اسماعیل مرادی
@esmaeeil-moradi700

میدBiometricPrompt با CryptoObject، passkeyها، Play Integrity، صفحه‌های حساس



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. اثر انگشت را به یک کلید گره بزنید

```
val spec = KeyGenParameterSpec.Builder(
    "payment_key", KeyProperties.PURPOSE_SIGN,
)
    .setAlgorithmParameterSpec(ECGenParameterSpec("secp256r1"))
    .setDigests(KeyProperties.DIGEST_SHA256)
    .setUserAuthenticationRequired(true)
    // timeout 0 = a fresh strong biometric for every use
    .setUserAuthenticationParameters(
        0, KeyProperties.AUTH_BIOMETRIC_STRONG) // API 30+
    .setInvalidatedByBiometricEnrollment(true)
    .setAttestationChallenge(enrollmentNonce)   // Q9
    .build()

KeyPairGenerator.getInstance(
    KeyProperties.KEY_ALGORITHM_EC, "AndroidKeyStore",
).apply { initialize(spec) }.generateKeyPair()
```

ثبت‌نام: کلید عمومی یک بار به سرور می‌رود. **هر پرداخت:** سرور یک challenge می‌فرستد، کاربر احراز هویت می‌کند، Keystore امضا می‌کند و سرور امضا را تأیید می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. امضا با CryptoObject

```
val signature = Signature.getInstance("SHA256withECDSA")
    .apply { initSign(privateKey) } // unusable until the prompt

val cb = object : BiometricPrompt.AuthenticationCallback() {
    override fun onAuthenticationSucceeded(
        result: BiometricPrompt.AuthenticationResult,
    ) {
        val s = checkNotNull(result.cryptoObject?.signature)
        s.update(serverChallenge)
        viewModel.submitPayment(s.sign())
    }
}
BiometricPrompt(activity, mainExecutor, cb)
    .authenticate(info, BiometricPrompt.CryptoObject(signature))
```

تلهٔ مصاحبه

«`setUserAuthenticationRequired(true)` *کافی است.*» — اگر timeout بیشتر از صفر باشد، کلید تا همان چند ثانیه بعد از **هر** باز شدن قفل کار می‌کند — بدون prompt شما. اثبات برای هر تراکنش به timeout صفر **و** یک `CryptoObject` نیاز دارد. زیر API 30 معادلش `setUserAuthenticationValidityDurationSeconds(-1)` است.

سؤال تکمیلی

«*کاربر یک اثر انگشت جدید اضافه کرد — حالا چه؟*» — `initSign()` خطای `KeyPermanentlyInvalidatedException` می‌دهد. هویت کاربر را از راه دیگری دوباره تأیید کنید، کلید جدید بسازید و دوباره ثبتش کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. passkey با Credential Manager

```
suspend fun signIn(activity: Activity, requestJson: String) {
    val request = GetCredentialRequest(
        listOf(GetPublicKeyCredentialOption(requestJson)),
    )
    try {
        val result = CredentialManager.create(activity)
            .getCredential(activity, request)
        val cred = result.credential
        if (cred is PublicKeyCredential) {
            api.finishLogin(cred.authenticationResponseJson)
        }
    } catch (e: GetCredentialCancellationException) {
        // user closed the sheet: stay quiet, no error dialog
    } catch (e: NoCredentialException) {
        showPasswordOrSignUp()
    }
}
```

- `requestJson` همان challenge مربوط به WebAuthn است که **از سرور شما** می‌آید؛ سرور پاسخ را هم تأیید می‌کند و بعد توکن‌های session را صادر می‌کند (روز ۱۲)
- همین API رمزهای عبور و Sign in with Google را هم در یک bottom sheet برمی‌گرداند، با `GetGoogleIdOption`



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. چرا passkey برنده است

	رمز عبور	passkey
سرور چه نگه می‌دارد	هش یک راز مشترک	**فقط یک کلید عمومی**
فیشینگ	کاربر می‌تواند آن را در هر سایتی تایپ کند	به دامنهٔ شما گره خورده — سایت جعلی چیزی نمی‌گیرد
اثر نشت دیتابیس	هش‌ها شکسته و دوباره استفاده می‌شوند	چیز قابل‌استفاده‌ای نشت نمی‌کند
تجربهٔ کاربر	به خاطر سپردن و تایپ کردن	باز کردن قفل دستگاه: اثر انگشت، چهره، پین

- اپ و دامنه را با **Digital Asset Links** و فایل `assetlinks.json` به هم وصل کنید — با اثر انگشت گواهی **app signing** از Play Console
- Google Sign-In قدیمی برای اندروید و Smart Lock for Passwords به نفع Credential Manager منسوخ شده‌اند

تلهٔ مصاحبه

«*با passkey اثر انگشت به سرور می‌رود.*» — هرگز. بیومتریک فقط کلید خصوصی را **روی خود دستگاه** باز می‌کند؛ سرور فقط امضایی روی challenge خودش دریافت می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. Play Integrity: گرفتن توکن

```
private val integrity =
    IntegrityManagerFactory.createStandard(app)
private var provider: StandardIntegrityTokenProvider? = null

// Once, early — preparing can take a while
suspend fun prepare() {
    provider = integrity.prepareIntegrityToken(
        PrepareIntegrityTokenRequest.builder()
            .setCloudProjectNumber(CLOUD_PROJECT_NUMBER)
            .build()
    ).await()
}

// Per high-value action, bound to that action's payload
suspend fun tokenFor(payload: String): String =
    checkNotNull(provider).request(
        StandardIntegrityTokenRequest.builder()
            .setRequestHash(sha256Hex(payload))
            .build()
    ).await().token()
```

اپ هیچ‌وقت verdict را نمی‌خواند — فقط توکن را همراه درخواست می‌فرستد. اگر provider منقضی شد، دوباره آماده‌اش کنید.

تلهٔ مصاحبه

«از *SafetyNet Attestation استفاده می‌کنیم.*» — بازنشسته شده. Play Integrity جایش را گرفته، با سیگنال‌های اپ، دستگاه و حساب کاربری در یک توکن.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. verdict به سرور شما چه می‌گوید

فیلد	معنا
`appIntegrity.appRecognitionVerdict`	`PLAY_RECOGNIZED` یعنی همان باینری‌ای که در پلی آپلود کردید
`deviceIntegrity.deviceRecognitionVerdict`	برچسب‌هایی مثل `MEETS_BASIC_INTEGRITY` و `MEETS_DEVICE_INTEGRITY` و `MEETS_STRONG_INTEGRITY`
`accountDetails.appLicensingVerdict`	`LICENSED` یعنی اپ از گوگل‌پلی نصب شده
`environmentDetails` (اختیاری)	اپ‌های پرخطر در حال اجرا؛ وضعیت Play Protect

تلهٔ مصاحبه

«*verdict را داخل اپ decode کن و مسدود کن.*» — توکن رمز و امضا شده؛ **سرور** آن را از طریق گوگل‌پلی decode می‌کند و تصمیم می‌گیرد. یک «if» سمت کلاینت با یک patch همیشه `true` می‌شود.

سؤال تکمیلی

«*کاربران بدون گوگل‌پلی چه؟*» — بدون Play services، verdictی در کار نیست: فروشگاه‌های جایگزین، بعضی کشورها. مسدود نکنید؛ مسیر جایگزین بگذارید: key attestation (سؤال ۹) و احراز تکمیلی.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. محافظت از صفحه‌های حساس

```
class RecoveryPhraseActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // No screenshots, recordings or recents thumbnail
        window.addFlags(WindowManager.LayoutParams.FLAG_SECURE)
        if (Build.VERSION.SDK_INT >= 31) {
            // requires HIDE_OVERLAY_WINDOWS (normal permission)
            window.setHideOverlayWindows(true)
        }
        setContent { RecoveryPhraseScreen() }
    }
}

// A secret dialog on an otherwise normal screen
Dialog(
    onDismissRequest = onDismiss,
    properties = DialogProperties(
        securePolicy = SecureFlagPolicy.SecureOn),
) { OtpContent() }
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. حمله‌ها و دفاع پلتفرم

حمله	دفاع پلتفرم
اسکرین‌شات، ضبط صفحه، cast	`FLAG_SECURE`؛ callback برای اسکرین‌شات از API 34 و برای ضبط صفحه از API 35
tapjacking — یک overlay کاربر را فریب می‌دهد تا جایی را لمس کند	از اندروید ۱۲ بیشتر لمس‌هایی که از overlay نامطمئن عبور می‌کنند مسدود می‌شوند؛ `filterTouchesWhenObscured`؛ پنهان کردن overlayها از API 31
بدافزار accessibility که صفحه را می‌خواند	`accessibilityDataSensitive` از API 34؛ تنظیمات محدودشده برای اپ‌های sideloadشده از اندروید ۱۳
کیبوردی که پین را یاد می‌گیرد	`inputType` از نوع password؛ `IME_FLAG_NO_PERSONALIZED_LEARNING`

تلهٔ مصاحبه

«*یک کیبورد امن اختصاصی بسازید.*» — توصیهٔ پارسال. کیبورد درون‌اپ فقط کد بیشتری برای اشتباه کردن است؛ همین flagهای پلتفرم جلوی یادگیری و ضبط را می‌گیرند.

تلهٔ مصاحبه

«`FLAG_SECURE` *روی Activity همه‌چیز را پوشش می‌دهد.*» — یک `Dialog` یا `PopupWindow` مبتنی بر View یک **پنجرهٔ جداست** و آن را به ارث نمی‌برد. دیالوگ‌های Compose به‌صورت پیش‌فرض به ارث می‌برند: `SecureFlagPolicy.Inherit`.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

اعتبارسنجی یکپارچگی روی سرور، key attestation، واکنش به دست‌کاری



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. اعتبارسنجی verdict یکپارچگی روی سرور

```
{
  "requestDetails": {
    "requestPackageName": "com.example.bank",
    "requestHash": "9f2c…",
    "timestampMillis": "1790000000000"
  },
  "appIntegrity": {
    "appRecognitionVerdict": "PLAY_RECOGNIZED",
    "certificateSha256Digest": ["6a6a…"]
  },
  "deviceIntegrity": {
    "deviceRecognitionVerdict": ["MEETS_DEVICE_INTEGRITY"]
  },
  "accountDetails": { "appLicensingVerdict": "LICENSED" }
}
```

۱. توکن را با API مربوط به `decodeIntegrityToken` در گوگل‌پلی decode کنید — **روی سرور**

۲. `requestPackageName` مال شما باشد؛ `timestampMillis` تازه باشد

۳. `requestHash` با هش **همان درخواستی که الان رسیده** برابر باشد — توکنِ «دیدن موجودی» نمی‌تواند «انتقال پول» را مجاز کند

۴. فقط بعد از این، verdictها را بخوانید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. تبدیل verdictها به سیاست

سیگنال‌ها	تصمیم
`PLAY_RECOGNIZED` همراه با `MEETS_DEVICE_INTEGRITY`	اجازه
فقط `MEETS_BASIC_INTEGRITY`، مثلاً bootloader باز	اجازه برای کارهای کم‌خطر؛ احراز تکمیلی برای انتقال
`UNRECOGNIZED_VERSION`	درخواست آپدیت از فروشگاه؛ مسدود کردن کارهای پرخطر
verdict دستگاه خالی یا نبودن گوگل‌پلی	برگشت به key attestation و OTP — شکست قطعی نه

تلهٔ مصاحبه

«*یکپارچگی را یک بار موقع ورود چک کن.*» — session چند روز طول می‌کشد؛ دستگاه در چند دقیقه عوض می‌شود. توکن را **در لحظهٔ کار پرارزش** بگیرید و با `requestHash` به همان کار گره بزنید.

سیاست را روی سرور نگه دارید

جدول بالا در تنظیمات بک‌اند زندگی می‌کند، نه در APK. وسط یک حمله بدون انتشار نسخهٔ جدید سخت‌گیرترش کنید — و یادتان باشد Play Integrity سقف درخواست روزانه دارد، پس فراخوانی‌ها را خرج کارهای مهم کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. key attestation: اثبات از خود سخت‌افزار امن

```
// Key created with setAttestationChallenge(nonce) — see Q4
val ks = KeyStore.getInstance("AndroidKeyStore")
    .apply { load(null) }
val chain = ks.getCertificateChain("payment_key").map {
    Base64.encodeToString(it.encoded, Base64.NO_WRAP)
}
api.registerKey(chain) // leaf certificate first
```

سرور این‌ها را تأیید می‌کند:

- زنجیره تا یک ریشهٔ attestation مورد اعتماد معتبر است، و هیچ گواهی‌ای در **فهرست ابطال** گوگل نیست
- **challenge** با nonceی که خودش صادر کرده برابر است
- `attestationSecurityLevel` برابر TEE یا StrongBox است — نه نرم‌افزار
- `attestationApplicationId` شامل package و گواهی امضای شماست
- `rootOfTrust` وضعیت verified boot و قفل بودن bootloader را نشان می‌دهد
- ویژگی‌های کلید: هدف SIGN، و الزام احراز هویت کاربر



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. attestation چه چیزی را ثابت می‌کند — و چه چیزی را نه

attestation از **خود KeyMint و Keystore** می‌آید، نه از Play services — برای همین جایگزین طبیعی جاهایی است که Play Integrity در دسترس نیست.

تلهٔ مصاحبه

«*attestation ثابت می‌کند دستگاه root نشده.*» — ثابت می‌کند **کلید** با ویژگی‌هایی که خواسته‌اید داخل سخت‌افزار امن است، و وضعیت boot را گزارش می‌کند. دربارهٔ اینکه چه چیز دیگری روی دستگاه اجرا می‌شود چیزی نمی‌گوید، و کلیدهای attestation نشت‌کرده هم وجود دارند — برای همین فهرست ابطال هست. یک سیگنال قوی است، نه اثبات.

تلهٔ مصاحبه

«*گواهی attestation ثابت می‌کند کد ما تغییر نکرده.*» — نسخهٔ پارسال چنین برداشتی داشت. این گواهی package و گواهی امضا را مشخص می‌کند — APK بسته‌بندی‌شدهٔ مجدد که با کلید دیگری امضا شده لو می‌رود؛ پروسس hookشده روی همان APK اصلی نه.

سؤال تکمیلی

«*چرا attestation را کنار نگذاریم و فقط به کلید عمومی ثبت‌شده اعتماد نکنیم؟*» — بدون attestation نمی‌توانید کلید وابسته به سخت‌افزار را از کلیدی که یک اپ patchشده — یا یک اسکریپت — در نرم‌افزار ساخته تشخیص بدهید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. دست‌کاری را تشخیص دادید — حالا چه؟

واکنش	چرا
سیگنال را گزارش کن، اپ را قابل‌استفاده نگه دار	سرور آن را با verdictهای یکپارچگی تطبیق می‌دهد؛ مهاجم نمی‌فهمد کدام چک فعال شد
احراز تکمیلی برای کارهای پرخطر: ورود دوباره، OTP، سقف کمتر	کاربران واقعی با دستگاه‌های غیرمعمول هم رد می‌شوند
واکنش را با تأخیر و متغیر انجام بده	کرش فوری مستقیم به محل چک اشاره می‌کند
مسدود کردن قطعی فقط با تصمیم **سرور**	یک جا برای تنظیم سیاست؛ بدون انتشار نسخهٔ جدید
kill switch و حداقل نسخه با remote config	واکنش به حملهٔ فعال در چند دقیقه

تلهٔ مصاحبه

«*اگر هش نخواند، فوراً اپ را ببند.*» — توصیهٔ پارسال. هشدارهای اشتباه را تنبیه می‌کند و دقیقاً به مهاجم یاد می‌دهد کجا را patch کند.

RASP تجاری

ابزارهایی در سطح DexGuard — obfuscation قوی‌تر، رمز کردن رشته‌ها، تشخیص hook — هزینهٔ حمله را بالا می‌برند و برای بانک یا بازی می‌ارزند. اما همچنان سمت کلاینت‌اند: آن‌ها را **علاوه بر** چک‌های سرور بخرید، نه به‌جایش.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

دفاع در عمق، OWASP MASVS و MASTG، امنیت زنجیرهٔ تأمین



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. دفاع در عمق برای یک جریان پرداخت

```
app ── payment + integrity token + signed challenge ──▶
backend
  ├─ decodeIntegrityToken ──▶ Google Play
  ├─ verify signature with the attested public key
  ├─ risk score = integrity + attestation + behavior
  └─ low: allow | medium: step up (OTP) | high: deny
```

- **سخت‌سازی:** R8، هیچ رازی در APK، درهای بسته‌شدهٔ بخش اول
- **سخت‌افزار:** کلیدهای Keystore گره‌خورده به احراز هویت، passkeyها
- **گواهی از راه دور:** Play Integrity و key attestation
- **سرور:** موتور ریسک، سقف‌ها، تشخیص ناهنجاری، لاگ ممیزی



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. اگر یک لایه بیفتد

لایهٔ دور زده‌شده	چه چیزی هنوز از کاربر محافظت می‌کند
obfuscation با R8 خوانده شد	هیچ رازی در APK نیست؛ قواعد سمت سرور
چک root هوک شد	verdict یکپارچگی روی سرور decode می‌شود
callback بیومتریک جعل شد	بدون کلید گره‌خورده به احراز هویت، امضایی در کار نیست
توکن session دزدیده شد	پرداخت هنوز یک challenge امضاشدهٔ تازه می‌خواهد
یکپارچگی در دسترس نیست	key attestation، احراز تکمیلی و سقف تراکنش

تلهٔ مصاحبه

«*pinning و تشخیص root داریم، پس امن هستیم.*» — هر لایه به‌تنهایی قابل دور زدن است. هدف طراحی این است که **هیچ دور زدن تکی‌ای پول جابه‌جا نکند.**

سؤال تکمیلی

«*چه چیزی را پایش می‌کنید؟*» — شکست‌های یکپارچگی به تفکیک نسخهٔ اپ و مدل دستگاه، نرخ احراز تکمیلی، شکست امضاها. جهش بعد از یک انتشار یعنی باگ؛ جهش از یک مزرعهٔ دستگاه یعنی حمله.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. OWASP MASVS: چک‌لیست مشترک

گروه MASVS	چه چیزی را پوشش می‌دهد	در این مجموعه
STORAGE	دادهٔ حساس ذخیره‌شده، لاگ، بکاپ	روز ۱۹
CRYPTO	Keystore، الگوریتم‌ها، مدیریت کلید	روز ۱۹
AUTH	passkey، بیومتریک، session	روز ۲۰ و ۱۲
NETWORK	TLS و pinning	روز ۱۲
PLATFORM	ارتباط بین پروسس‌ها، intent، WebView، نشت از UI	روز ۱۹ و ۲۰
CODE	وابستگی‌ها، اعتبارسنجی ورودی، آپدیت	روز ۲۰
RESILIENCE	obfuscation، یکپارچگی، ضد دست‌کاری	روز ۲۰
PRIVACY	حداقل‌سازی داده، شفافیت، کنترل کاربر	روز ۱۹

MASVS می‌گوید چه چیزی باید محقق شود. **MASTG** (راهنمای تست) می‌گوید *چطور* تستش کنید، با تکنیک‌ها و ابزارها. MASWE هم ضعف‌های مشخص بین این دو را فهرست می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. استفادهٔ عملی

- یک **testing profile** انتخاب کنید: L1 پایهٔ همهٔ اپ‌ها، L2 برای دادهٔ حساس مثل سلامت و مالی، R برای مقاومت در برابر مهندسی معکوس، P برای حریم خصوصی
- کنترل‌ها را به backlog و **Definition of Done** وصل کنید — نه به یک PDF که سالی یک بار خوانده می‌شود
- با MASTG تست کنید: تحلیل ایستا و پویا، jadx، Frida، MobSF
- اثبات مستقل می‌خواهید؟ برنامهٔ **MASA** در App Defense Alliance اپ را بر اساس MASVS ارزیابی می‌کند؛ گوگل‌پلی می‌تواند نتیجه را در بخش Data safety نشان بدهد

تلهٔ مصاحبه

«*MASVS سطح‌های L1 و L2 دارد.*» — قدیمی است. MASVS v2 فقط فهرست کنترل‌هاست؛ سطح‌ها به **testing profileهای** MAS منتقل شده‌اند.

تلهٔ مصاحبه

«*Resilience ضعف ذخیره‌سازی را جبران می‌کند.*» — پروفایل R **روی** L1 و L2 می‌آید، هرگز به‌جای آن‌ها. obfuscate کردن کدی که توکن را در logcat می‌نویسد فقط باگ را از چشم خودتان پنهان می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. زنجیرهٔ تأمین: build شما هم سطح حمله است

```
// settings.gradle.kts
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        // Internal groups resolve ONLY from the internal repo
        exclusiveContent {
            forRepository {
                maven("https://maven.internal.example.com")
            }
            filter { includeGroupByRegex("com\\.example\\..*") }
        }
    }
}
```

```
# Record checksums; the build fails when an artifact changes
./gradlew --write-verification-metadata sha256 help
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. جعبه‌ابزار زنجیرهٔ تأمین

ریسک	کنترل
dependency confusion	فیلتر کردن repository با `exclusiveContent`
artifact دست‌کاری‌شده	dependency verification در Gradle با `verification-metadata.xml`
نسخه‌های دارای آسیب‌پذیری شناخته‌شده	Dependabot یا Renovate، OSV-Scanner، هشدارهای Play SDK Index
Gradle wrapper مخرب	`distributionSha256Sum` همراه با اعتبارسنجی wrapper در CI
نشت کلید امضا	Play App Signing: کلید امضای اپ دست گوگل است؛ کلید آپلود گم‌شده قابل ریست است
نداشتن فهرست وابستگی‌ها	یک SBOM (مثلاً CycloneDX) برای هر انتشار

تلهٔ مصاحبه

«*وابستگی‌ها کد دیگران‌اند، پس مشکل دیگران‌اند.*» — با UID و مجوزهای شما اجرا می‌شوند (روز ۱۹). پلاگین‌های Gradle هم کدند — و روی CI، کنار رازهای امضای شما اجرا می‌شوند.

جواب سینیور در یک جمله

«چک‌های کلاینت هزینه را بالا می‌برند، کلیدهای سخت‌افزاری اثبات می‌سازند، سرور تصمیم می‌گیرد — و buildی که همهٔ این‌ها را تولید می‌کند خودش بخشی از سطح حمله است.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۲۰ — خلاصهٔ یک‌صفحه‌ای

- R8 کوچک می‌کند و تغییر نام می‌دهد؛ رشته‌ها را رمز نمی‌کند. هیچ رازی در APK نگذارید.
- چک root و دست‌کاری سیگنال قابل دور زدن است: گزارشش کنید، با آن کرش نکنید.
- با Play App Signing با گواهی **app signing** مقایسه کنید، نه کلید آپلود.
- بیومتریک بدون `CryptoObject` فقط یک callback است؛ به کلید Keystore با timeout صفر گره‌اش بزنید.
- passkey با Credential Manager: فقط کلید عمومی روی سرور، ضد فیشینگ، با Digital Asset Links.
- Play Integrity جای SafetyNet را گرفت: توکن با `requestHash` ، و decode و تصمیم **روی سرور**.
- گوگل‌پلی نیست؟ key attestation و احراز تکمیلی — با چک فهرست ابطال.
- صفحهٔ حساس: `FLAG_SECURE` ، محافظت از overlay، `accessibilityDataSensitive` ؛ بدون کیبورد اختصاصی.
- MASVS یعنی «چه»، MASTG یعنی «چطور»؛ L1 و L2 و R و P حالا testing profile هستند.
- زنجیرهٔ تأمین: فیلتر repository، dependency verification، checksum برای wrapper، Play App Signing و SBOM.

روز ۲۱ در راه است: Build System و Gradle، بخش ۱. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700